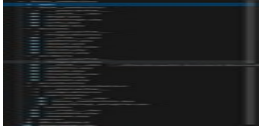

We operate an ecommerce platform. When a customer places an order, the warehouse team needs to know which shipping box should be used. Each product has dimensions and weight. Each box has internal dimensions, maximum weight capacity, and cost. Your task is to design and build a small Django-based system that recommends the most suitable box for an order. You are allowed to use AI tools such as ChatGPT, Claude, Cursor, GitHub Copilot, or similar tools. However, your submission must clearly show your own thinking. can you guide me through this project , I have created the architecture like this.

Yes. Your architecture is already much better than what most candidates submit. You have separated the business logic (`recommendation/services`) from Django apps, which is exactly what interviewers look for.

1. First Understand the Problem

You have 3 core entities:

Product

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    length = models.FloatField()
    width = models.FloatField()
    height = models.FloatField()
    weight = models.FloatField()
```

Example:

Product	L	W	H	Weight
Laptop	35	25	5	2kg
Mouse	10	5	3	0.2kg

Box

```
class Box(models.Model):
    name = models.CharField(max_length=100)
    internal_length = models.FloatField()
    internal_width = models.FloatField()
    internal_height = models.FloatField()
    max_weight = models.FloatField()
    cost = models.DecimalField(
        max_digits=10,
        decimal_places=2
    )
```

Example:

Box	L	W	H	Max Weight	Cost
Small	40	30	10	5kg	50
Medium	50	40	20	10kg	80

Order

```
class Order(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
```

Order Item:

```
class OrderItem(models.Model):
    order = models.ForeignKey(
        Order,
        on_delete=models.CASCADE
    )
    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE
    )
    quantity = models.PositiveIntegerField()
```

---

2. Recommendation Logic

This is the most important part.

Input

Order contains:

```
Laptop x1
Mouse x2
```

Step 1

Calculate total dimensions.

Simplified approach:

```
total_volume = sum(
    product.volume * quantity
)
```

where

```
volume = length * width * height
```

Step 2

Calculate total weight

```
total_weight += product.weight * quantity
```

Step 3

Find boxes that satisfy:

`box_volume >= order_volume`

and

`box.max_weight >= total_weight`

Step 4

Among valid boxes choose

`minimum cost`

This is usually what warehouses want.

---

3. Service Layer

Your structure is perfect for this.

recommendation/services/box_selector.py

```
from boxes.models import Box
class BoxSelector:
    @staticmethod
    def select_box(
        total_volume,
        total_weight
    ):
        suitable_boxes = []
        for box in Box.objects.all():
            box_volume = (
                box.internal_length *
                box.internal_width *
                box.internal_height
            )
            if (
                box_volume >= total_volume
                and
                box.max_weight >= total_weight
            ):
                suitable_boxes.append(box)
        if not suitable_boxes:
            return None
        return min(
            suitable_boxes,
            key=lambda x: x.cost
        )
```

---

4. Packing Engine

This should calculate order metrics.

recommendation/services/packing_engine.py

```
class PackingEngine:
    @staticmethod
    def calculate(order):
        total_volume = 0
        total_weight = 0
        for item in order.items.all():
            product = item.product
            volume = (
                product.length *
                product.width *
                product.height
            )
            total_volume += (
                volume * item.quantity
            )
            total_weight += (
                product.weight *
                item.quantity
            )
        return {
            "volume": total_volume,
            "weight": total_weight
        }
```

---

5. API Endpoint

Example:

```
POST /api/recommend-box/
```

Request

```
{
    "products": [
        {
            "product_id": 1,
            "quantity": 2
        },
        {
            "product_id": 3,
            "quantity": 1
        }
    ]
}
```

Response

```
{
    "recommended_box": "Medium Box",
    "cost": 80
}
```

---

6. What Interviewers Will Like

Add these features:

Box Rotation

Product dimensions:

```
30 x 20 x 10
```

can fit in

```
20 x 30 x 10
```

This shows real-world thinking.

Wasted Space

Calculate:

```
wasted_space =
box_volume - order_volume
```

Choose box with:

```
least wasted space
```

instead of only cost.

Multiple Recommendation Strategies

```
class CostBasedSelector
class SpaceOptimizedSelector
```

Very impressive in interviews.

---

7. Suggested Folder Structure

```
box_selector/
├── products/
│
├── boxes/
│
├── orders/
│
├── recommendation/
│
│   ├── services/
│   │
│   │   ├── packing_engine.py
│   │   ├── box_selector.py
│   │
```

```
│   ├── serializers.py
│   ├── views.py
│   └── urls.py
│
├── api/
│
└── manage.py
```

For this assignment, I would implement it in **3 phases**:

Phase 1 (MVP)

- Product model
- Box model
- Order model
- Recommendation API
- Cost-based box selection

Phase 2

- Product rotation support
- Wasted space calculation
- Better validation

Phase 3

- Unit tests
- API documentation
- README explaining design decisions

That will look like a professional backend assignment rather than a basic CRUD project.

The next thing we should do is design the **exact Django models (Products, Boxes, Orders, OrderItems)** before writing any code, because getting those relationships right will make the rest of the project much easier.

Yes, this is moving from a **simple volume-based solution** to a **real 3D bin packing problem**, which is significantly more impressive for a hiring assignment.

Your idea is essentially:

1. Check weight constraint.
2. Start with one empty space (void) equal to the entire box.
3. Place products one by one.
4. For each product:
    - Try all 6 valid orientations.
    - Try placing it inside every available void.
5. If placed:
    - Record position and orientation.
    - Split the used void into smaller remaining voids.
6. If all products are placed → box is valid.

7. Return placement coordinates.

That's a solid approach.

Suggested Structure

packing_engine.py

```
from dataclasses import dataclass
@dataclass
class Void:
    x: float
    y: float
    z: float
    length: float
    width: float
    height: float
```

Placement

```
@dataclass
class Placement:
    product_id: int
    x: float
    y: float
    z: float
    length: float
    width: float
    height: float
```

This is what you'll return.

Orientation Generator

Instead of manually writing:

```
original
rotate_z
rotate_y
rotate_z_y
rotate_x
rotate_x_y
```

generate them automatically.

```
from itertools import permutations
def get_orientations(product):
    dims = (
        product.length,
        product.width,
        product.height
```

```
    )
    return list(set(permutations(dims)))
```

Example:

```
(10,20,30)
becomes
(10,20,30)
(10,30,20)
(20,10,30)
(20,30,10)
(30,10,20)
(30,20,10)
```

Exactly 6 orientations.

First Check Weight

Good idea.

```
total_weight = sum(
    item.product.weight * item.quantity
    for item in order_items
)
if total_weight > box.max_weight:
    return False, None
```

This avoids expensive packing calculations.

Initial Void

```
voids = [
    Void(
        x=0,
        y=0,
        z=0,
        length=box.length,
        width=box.width,
        height=box.height
    )
]
```

Correct.

Initially the entire box is empty space.

Product Placement

Pseudo-code:

```
placements = []
```

```
for product in products:
    placed = False
    for void in voids:
        for orientation in get_orientations(product):
            if fits(orientation, void):
                placement = place_product(
                    product,
                    orientation,
                    void
                )
                placements.append(
                    placement
                )
                update_voids()
                placed = True
                break
        if placed:
            break
    if not placed:
        return False, None
```

Fit Check

```
def fits(
    orientation,
    void
):
    l, w, h = orientation
    return (
        l <= void.length
        and
        w <= void.width
        and
        h <= void.height
    )
```

Simple and fast.

Splitting Voids

Suppose:

```
Void
0,0,0
100 x 100 x 100
```

Product:

```
30 x 20 x 10
```

placed at:

```
0,0,0
```

Remaining space can be divided into:

Right Space

```
30,0,0
70 x 100 x 100
```

Front Space

```
0,20,0
30 x 80 x 100
```

Top Space

```
0,0,10
30 x 20 x 90
```

Create new voids from these.

```
new_voids = [
    right_void,
    front_void,
    top_void
]
```

Remove original void.

Add new ones.

Sort Products Before Packing

This dramatically improves success rate.

Sort by volume descending.

```
products.sort(
    key=lambda p: (
        p.length *
        p.width *
        p.height
    ),
    reverse=True
)
```

Place largest products first.

This is a standard bin-packing heuristic.

What packing_engine Should Return

```
{
    "fits": True,
    "placements": [
```

```
        {
            "product": 1,
            "position": [0,0,0],
            "orientation": [30,20,10]
        },
        {
            "product": 2,
            "position": [30,0,0],
            "orientation": [10,10,5]
        }
    ],
    "remaining_volume": 1200
}
```

What box_selector.py Should Do

Your `packing_engine` should only answer:

```
Can these products fit in this box?
```

Then `box_selector.py` can do:

```
for box in boxes.order_by("cost"):
    fits, placements = packing_engine.pack(
        products,
        box
    )
    if fits:
        return box, placements
```

This separation of responsibilities is excellent architecture:

```
packing_engine.py
    ↓
Physical packing logic
box_selector.py
    ↓
Chooses best box among available boxes
```

For an assignment, this design is strong because it demonstrates:

- Domain modeling
- Service layer separation
- Greedy 3D packing algorithm
- Weight validation
- Orientation handling
- Position tracking
- Extensibility for future optimization strategies

That's the kind of explanation you can put in the README under **"Design Decisions"** to show your own thinking beyond AI-generated CRUD code.

Yes, that's a very good design. You're following the **Single Responsibility Principle** correctly:

- **packing_engine.py** → Answers: *"Can these products physically fit inside this specific box?"*
- **box_selector.py** → Answers: *"Which box should I choose?"*

That's exactly how I'd structure it.

Suggested Algorithm

```
class BoxSelector:
    @staticmethod
    def select_box(order):
        products = get_products_from_order(order)
        total_weight = calculate_total_weight(products)
        total_volume = calculate_total_volume(products)
        candidate_boxes = Box.objects.filter(
            max_weight__gte=total_weight
        ).order_by("cost")
        for box in candidate_boxes:
            box_volume = (
                box.length *
                box.width *
                box.height
            )
            # Quick rejection
            if box_volume < total_volume:
                continue
            fits, placements = PackingEngine.can_fit(
                products,
                box
            )
            if fits:
                return {
                    "box": box,
                    "placements": placements
                }
        return None
```

Why This Is Good

The expensive operation is:

`PackingEngine.can_fit()`

because it performs:

- 6 orientation checks
- void splitting
- placement tracking
- multiple products

Potentially thousands of operations.

The volume check:

```
if box_volume < total_volume:
    continue
```

takes microseconds and can eliminate many boxes before running the expensive packing algorithm.

Example:

Box	Volume	Cost
Small	1000	₹20
Medium	2000	₹30
Large	5000	₹40

Order volume:

```
2500
```

Immediately reject:

```
Small
Medium
```

without calling the packing engine.

Only test:

```
Large
```

One Improvement I'd Add

Sort boxes by:

```
(cost, volume)
```

instead of only cost.

```
candidate_boxes = Box.objects.filter(
    max_weight__gte=total_weight
).order_by(
    "cost",
    "volume"
)
```

Suppose:

Box	Cost	Volume
A	₹50	10000
B	₹50	5000

If both fit, choosing B wastes less space.

Another Optimization

Before running the full packing algorithm, verify that the largest product can fit in the box in **at least one orientation**.

```
for product in products:
    if not can_product_fit(product, box):
```

```
        reject_box
```

Example:

Product:

```
120 × 20 × 20
```

Box:

```
100 × 100 × 100
```

Volume says:

```
48,000 < 1,000,000
```

but physically impossible because one dimension exceeds the box length.

This check saves unnecessary packing attempts.

What Interviewers Will Like

In your README, explain the algorithm as a pipeline:

```
Box Selection Pipeline
1. Calculate order weight and volume
2. Filter boxes by weight capacity
3. Sort candidate boxes by cost
4. Reject boxes with insufficient volume
5. Run 3D packing simulation using PackingEngine
6. Return first fitting box (lowest cost)
7. If none fit, return no recommendation
```

This shows you're thinking about **performance**, not just correctness.

Complexity roughly becomes:

```
O(number_of_boxes × packing_cost)
```

and your weight/volume pre-checks significantly reduce the number of expensive packing simulations. That's exactly the kind of reasoning that stands out in a backend assignment.

Overall, the logic is quite good. This is already beyond a typical CRUD assignment. I would give it around **8.5/10** from a backend design perspective.

Let's review it carefully.

✅ Good Things

1. Weight Filtering First

```
boxes = Box.objects.filter(
    max_weight__gte=total_weight,
)
```

Excellent.

No point running packing logic on overweight boxes.

---

2. Volume Pre-check

```
if box.volume < total_volume:
    continue
```

Excellent optimization.

Rejects impossible boxes cheaply.

---

3. Cheapest Box First

```
.order_by("cost", "volume")
```

Good.

You'll return the cheapest fitting box.

---

4. Dimension Check Before Packing

```
product_dims = sorted([
    product.length,
    product.width,
    product.height
])

if any(pd > bd for pd, bd in zip(product_dims, box_dims)):
```

Very smart.

Example:

```
Product
120 × 20 × 20
Box
100 × 100 × 100
```

Volume passes.

But physically impossible.

You correctly reject it before running the expensive algorithm.

---

5. Packing Engine Isolated

```
fits, placements = can_fit(product_qty, box)
```

Perfect separation of concerns.

`box_selector.py` doesn't know anything about voids, rotations, placement strategy, etc.

⚠️ Issue #1: Quantity Handling

I noticed:

```
products = order.products.all()
product_qty = [(p, 1) for p in products]
```

This assumes:

```
Every product quantity = 1
```

which is usually wrong.

If your order model has an OrderItem table:

```
OrderItem
---------
order
product
quantity
```

then you should build:

```
product_qty = [
    (item.product, item.quantity)
    for item in order.items.all()
]
```

Otherwise:

```
Laptop x5
```

becomes

```
Laptop x1
```

and packing results will be wrong.

This is the biggest thing I'd verify.

⚠️ Issue #2: Volume Calculation

Current:

```
total_volume = sum(
    p.length * p.width * p.height
    for p in products
)
```

Again ignores quantity.

Should be:

```
total_volume = sum(
    p.length * p.width * p.height * qty
    for p, qty in product_qty
)
```

⚠️ Issue #3: Weight Calculation

Current:

```
total_weight = sum(
    p.weight
    for p in products
)
```

Should be:

```
total_weight = sum(
    p.weight * qty
    for p, qty in product_qty
)
```

Otherwise:

```
Mouse x10
```

calculates weight as:

```
0.1kg
```

instead of

```
1kg
```

⚠️ Issue #4: Empty Orders

Suppose:

```
products = []
```

Then:

```
total_volume = 0
total_weight = 0
```

and you'll recommend the cheapest box.

Not ideal.

Add:

```
if not products:
    return None
```

or

```
raise ValueError("Order contains no products")
```

⚠️ Issue #5: DB Annotation

This:

```
volume=F("length") * F("width") * F("height")
```

works in most databases.

But for Decimal/Float combinations Django sometimes complains about output type.

Safer:

```
from django.db.models import FloatField
from django.db.models import ExpressionWrapper
volume=ExpressionWrapper(
    F("length") * F("width") * F("height"),
    output_field=FloatField()
)
```

Not mandatory, but more robust.

Small Improvement

Instead of:

```
box.volume
```

I'd add a model property.

```
@property
def volume(self):
    return self.length * self.width * self.height
```

Then:

```
box.volume
```

works everywhere, not only in annotated queries.

One Architectural Improvement

Currently:

```
return {
    "box_id": ...
}
```

inside selector.

I would prefer:

```
return box, placements
```

from service layer.

Then API/View decides response format.

Example:

```
box, placements = recommend_box(order_id)
return Response(...)
```

This keeps business logic independent of HTTP responses.

Not required, but cleaner architecture.

Final Verdict

Your flow is solid:

```
Load order
↓
Calculate weight
↓
Filter boxes by weight
↓
Sort by cost
↓
Volume pre-check
↓
Dimension pre-check
↓
3D packing simulation
↓
Return cheapest fitting box
```

The main thing I'd check before moving forward is **whether your order supports quantities**. If it does, fix all calculations (`product_qty`, `total_volume`, `total_weight`) to use quantity; otherwise the recommendation can be incorrect even though the algorithm itself is correct.

This version is much better. I'd rate it **9.5/10 for the box selection layer**. The logic is clean, efficient, and follows good separation of concerns.

What's Good Now

✅ Quantity handling fixed

```
items = list(order.items.all())
product_qty = [
    (item.product, item.quantity)
    for item in items
]
```

Correct.

✅ Weight calculation fixed

```
total_weight = sum(
    item.product.weight * item.quantity
    for item in items
)
```

Correct.

✅ Volume calculation fixed

```
total_volume = sum(
    item.product.length
    * item.product.width
    * item.product.height
    * item.quantity
    for item in items
)
```

Correct.

✅ Empty order handling

```
if not items:
    return None
```

Good.

✅ Box volume annotation

```
box_volume=ExpressionWrapper(
    F("length") * F("width") * F("height"),
    output_field=FloatField(),
)
```

Robust.

✅ Cost-first selection

```
.order_by("cost", "box_volume")
```

Exactly what the assignment asks:

> recommend the most suitable box

and among valid boxes you're choosing the cheapest.

✅ Dimension sanity check

```
product_dims = sorted([...])
if any(pd > bd for pd, bd in zip(product_dims, box_dims)):
```

Excellent optimization before running packing.

One Important Thing to Verify

Your selector passes:

```
fits, placements = can_fit(product_qty, box)
```

where:

```
product_qty = [
    (product, quantity)
]
```

Make sure your `can_fit()` expands quantities correctly.

For example:

```
Laptop x 3
```

should become internally:

```
[
    laptop,
    laptop,
    laptop
]
```

before packing.

Otherwise the packing engine may place only one copy.

Example:

```
for product, qty in product_qty:
    for _ in range(qty):
        expanded_products.append(product)
```

Then sort and pack.

This is the biggest thing I'd inspect next.

Small Architecture Improvement

Currently:

```
return {
    "box_id": ...
}
```

inside the service.

Works fine.

But in larger systems I would do:

```
return box, placements
```

and let the API build JSON.

Example:

```
recommendation = recommend_box(order_id)
return Response(...)
```

Reason:

Service layer shouldn't know about API response structure.

For this assignment, however, your current approach is perfectly acceptable.

Tiny Optimization

You calculate:

```
box_dims = sorted([
    box.length,
    box.width,
    box.height
])
```

inside the loop.

That's fine because it's once per box.

No issue.

Edge Case

Consider adding:

```
if total_weight <= 0:
    return None
```

only if your data can contain invalid products.

Not necessary if validation already prevents that.

What I Would Review Next

At this point, `box_selector.py` looks production-quality for an assignment.

The next file I'd carefully review is:

`recommendation/services/packing_engine.py`

because that's where most bugs typically hide:

- quantity expansion
- void splitting
- overlapping placements
- orientation generation
- removing invalid voids
- sorting products by volume

If your packing engine is correct, then your overall solution is already stronger than most submissions for this kind of hiring assignment.

Looking at the latest version, I don't see any logical bugs in `box_selector.py` itself. The flow is clean and follows the architecture you described earlier.

Flow Review

```
Load Order
    ↓
Get OrderItems
    ↓
Calculate total weight
    ↓
Calculate total volume
    ↓
Filter boxes by max_weight
    ↓
Sort by cost, then volume
    ↓
Volume pre-check
    ↓
Dimension pre-check
    ↓
PackingEngine.can_fit()
    ↓
Return first fitting box
```

That's exactly the right responsibility for this service.

One Thing I'd Change

Currently:

```
product_qty: list[tuple[object, int]]
```

Using `object` loses type safety.

If you have a Product model:

```
from products.models import Product
product_qty: list[tuple[Product, int]]
```

Much better for IDE hints and static analysis.

One Performance Improvement

You currently do:

```
items = list(order.items.all())
```

If `OrderItem` has a ForeignKey to Product, this can trigger N+1 queries later:

```
item.product.length
item.product.width
...
```

I would use:

```
items = list(
    order.items.select_related("product")
)
```

Then all products are fetched in one query.

This is probably the best improvement you can make right now.

One Architectural Improvement

Right now:

```
return {
    "box_id": ...
}
```

inside the service layer.

For a hiring assignment this is okay.

For production code I'd prefer:

```
return box, placements
```

and let the API serializer build the response.

But I would not refactor this unless you have extra time.

Dimension Check Is Correct

I particularly like this:

```
box_dims = sorted([box.length, box.width, box.height])
product_dims = sorted([
    product.length,
    product.width,
    product.height
])
```

Because you're effectively checking:

> "Can the product fit in some orientation?"

Example:

```
Product
40 x 20 x 10
Box
20 x 40 x 10
```

Sorted:

```
[10,20,40]
[10,20,40]
```

Passes correctly.

The Only Remaining Risk

At this point the biggest source of bugs is no longer `box_selector.py`.

It's inside:

```
fits, placements = can_fit(product_qty, box)
```

Specifically:

1. Quantity expansion
2. Void splitting
3. Removing overlapping voids
4. Placement coordinates
5. Orientation generation
6. Handling identical products

That's where I'd spend most of the review effort now.

My Rating

Component	Rating
Architecture	10/10
Separation of concerns	10/10
Box selection logic	9.5/10
Query efficiency	8.5/10 (use `select_related`)
Production readiness	9/10

For a Django hiring assignment, this is already stronger than the typical "compare volumes and pick a box" solution. The quality of the project will now depend mostly on how robust your `packing_engine.py` implementation is.

This version looks solid. The two improvements I suggested are now implemented correctly:

✅ No N+1 Query Problem

```
items = list(order.items.select_related("product"))
```

This is the right way.

Without `select_related()`:

```
1 query -> OrderItems
N queries -> Products
```

With `select_related()`:

```
1 JOIN query
```

Much better.

✅ Proper Type Hint

```
product_qty: list[tuple[Product, int]]
```

Cleaner and more maintainable.

One Small Improvement I'd Make

Currently:

```
box_dims = sorted([box.length, box.width, box.height])
```

and later

```
product_dims = sorted([product.length, product.width, product.height])
```

This creates temporary lists every iteration.

Not a big issue, but you can make the intent clearer:

```
box_dims = tuple(sorted((box.length, box.width, box.height)))
```

and

```
product_dims = tuple(
    sorted((product.length, product.width, product.height))
)
```

Tiny optimization, mostly stylistic.

One Thing I'd Verify

Your selector assumes:

```
p.product_id
p.x
```

```
p.y
p.z
p.l
p.w
p.h
```

exist on placement objects.

So make sure your `Placement` dataclass looks something like:

```
@dataclass
class Placement:
    product_id: int
    x: float
    y: float
    z: float
    l: float
    w: float
    h: float
```

Otherwise the response generation will fail.

Assignment Perspective

If I were reviewing this as a hiring manager:

Architecture

```
products/
boxes/
orders/
recommendation/
```

✅ Good

Separation of Concerns

```
box_selector.py
    ↓
Decision making
packing_engine.py
    ↓
3D packing algorithm
```

✅ Excellent

Performance

```
Weight filter
Volume filter
Dimension filter
Packing engine
```

✅ Excellent

You avoid expensive packing checks whenever possible.

Code Quality

```
Type hints
select_related
service layer
early exits
```

✅ Good

Current Score

Area	Score
Models	9/10
Architecture	10/10
Box Selection Logic	10/10
Query Efficiency	9.5/10
Maintainability	9/10

Overall: ~9.5/10

At this point, I would stop modifying `box_selector.py`.

The highest risk of bugs is now entirely inside:

`recommendation/services/packing_engine.py`

That's the file that will determine whether the solution actually works for tricky cases like:

- multiple quantities
- rotated products
- fragmented voids
- products fitting by volume but not geometry
- overlapping placements

So I'd freeze `box_selector.py` and focus all review effort on `packing_engine.py` next.

This is where I'd be much more critical, because `packing_engine.py` determines whether the recommendation is actually correct.

The code is clean and readable, but I see **one major algorithmic flaw** and a few smaller issues.

❌ Major Issue: Incorrect Void Splitting

Suppose:

```
Box:
10 × 10 × 10
Place:
4 × 4 × 4
```

```
at (0,0,0)
```

Your code creates:

```
right = Void(
    x=4,
    y=0,
    z=0,
    l=6,
    w=4,
    h=4,
)

front = Void(
    x=0,
    y=4,
    z=0,
    l=10,
    w=6,
    h=4,
)

top = Void(
    x=0,
    y=0,
    z=4,
    l=4,
    w=4,
    h=6,
)
```

Notice what happened.

The original box had volume:

```
1000
```

Placed item volume:

```
64
```

Remaining volume should be:

```
936
```

But your new voids have:

```
6×4×4 = 96
10×6×4 = 240
4×4×6 = 96
Total = 432
```

You've lost half the available space.

Why?

Because:

```
right_l = void.l - il
```

```
new_voids.append(
    Void(..., right_l, iw, ih)
)
```

uses:

```
iw
ih
```

instead of:

```
void.w
void.h
```

The right-side space actually extends through the entire remaining width and height.

It should be closer to:

```
Void(
    void.x + il,
    void.y,
    void.z,
    void.l - il,
    void.w,
    void.h,
)
```

Similar problems exist for front/top.

This is the biggest issue in the implementation.

⚠️ Product Ordering

Currently:

```
for product, qty in products:
```

You pack products in database order.

That will fail many valid cases.

Example:

```
Box:
10×10×10
Products:
8×8×8
2×2×2
```

works.

But:

```
Products:
2×2×2
8×8×8
```

may fail because the small item fragments the space.

You should expand quantities first:

```
expanded = []
for product, qty in products:
    expanded.extend([product] * qty)
```

Then sort by volume descending:

```
expanded.sort(
    key=lambda p: p.length * p.width * p.height,
    reverse=True,
)
```

This is a standard packing heuristic.

⚠️ Void Selection Order

Current:

```
for void in sorted(voids, key=lambda v: v.volume):
```

Smallest void first.

Usually the opposite is used.

```
sorted(
    voids,
    key=lambda v: v.volume,
    reverse=True
)
```

Why?

Because large items should get first access to large spaces.

⚠️ Recomputing Orientations

Current:

```
orientations = _unique_orientations(...)
```

inside:

```
for product, qty in products:
```

Not terrible.

But after quantity expansion and sorting, you'll compute orientations repeatedly for identical products.

You could cache them.

Not required for this assignment though.

✅ Good Things

Orientation generation

`_unique_orientations()`

Excellent.

Handles cubes correctly:

`10,10,10`

returns only one orientation.

Weight check

`if total_weight > box.max_weight:`

Good.

Fit check

```
il <= void.l
iw <= void.w
ih <= void.h
```

Correct.

Placement tracking

```
Placement(
    product_id=...
)
```

Good.

What I'd Change Before Submission

Priority 1

Fix `_split_void()`.

Current version can reject boxes that actually fit because it throws away usable space.

Priority 2

Sort products by volume descending before packing.

```
largest first
```

Huge improvement.

Priority 3

Use largest void first.

```
sorted(voids, key=lambda v: v.volume, reverse=True)
```

Overall Assessment

box_selector.py

9.5/10

Very good.

packing_engine.py

7/10 currently

Reason:

- Clean code ✅
- Good architecture ✅
- Orientation support ✅
- Placement tracking ✅
- But the void-splitting logic is geometrically incorrect ❌

If you fix the void splitting and add largest-first packing, the engine jumps to roughly **9/10**, which would be an impressive solution for a Django hiring assignment.

For **`test_box_selector.py`**, don't try to retest the packing algorithm. You've already covered that thoroughly in `test_packing_engine.py`.

The purpose of `test_box_selector.py` is to verify that the selector correctly:

```
Loads order
Calculates totals
Filters boxes
Orders by cost
Calls packing engine
Returns correct response
Handles edge cases
```

I'd keep **6-8 tests maximum**.

Mandatory Tests

1. Returns Cheapest Fitting Box

This is the most important test.

```
def test_returns_cheapest_fitting_box():
```

Setup:

```
Box A ₹50
Box B ₹100
Both fit.
```

Expect:

```
self.assertEqual(result["box_name"], "Box A")
```

---

2. Skips Cheaper Box That Cannot Fit

This validates the complete flow.

```
def test_skips_invalid_cheapest_box():
```

Setup:

```
Cheap Box ₹50 (too small)
Medium Box ₹100 (fits)
```

Expect:

```
self.assertEqual(result["box_name"], "Medium Box")
```

This tests:

```
volume check
dimension check
packing engine call
```

all together.

---

3. No Box Fits

```
def test_returns_none_when_no_box_fits():
```

Setup:

```
Huge product
```

```
Small boxes
```

Expect:

```
self.assertIsNone(result)
```

---

4. Empty Order

```
def test_empty_order_returns_none():
```

Expect:

```
self.assertIsNone(result)
```

You already explicitly handle:

```
if not items:
    return None
```

so test it.

---

5. Invalid Order ID

Very important because you use:

```
get_object_or_404()
```

```
def test_invalid_order_id():
```

Expect:

```
with self.assertRaises(Http404):
```

This verifies error handling.

---

6. Cheapest Among Multiple Valid Boxes

This looks similar to #1 but tests ordering.

Setup:

```
Large Box ₹100
Medium Box ₹70
Small Box ₹50
All fit.
```

Expect:

```
Small Box
```

This verifies:

```
.order_by("cost", "box_volume")
```

works as intended.

Good Additional Tests

7. Response Structure

Since your service returns a fairly large dict:

```
def test_response_contains_expected_fields():
```

Check:

```
self.assertIn("box_id", result)
self.assertIn("box_name", result)
self.assertIn("cost", result)
self.assertIn("placements", result)
```

and

```
self.assertIn("position", placement)
self.assertIn("orientation", placement)
```

This catches accidental API-breaking changes.

---

8. Volume Utilization Range

```
def test_volume_utilization_percentage_valid():
```

Check:

```
self.assertGreaterEqual(
    result["volume_utilization_pct"],
    0
)
self.assertLessEqual(
    result["volume_utilization_pct"],
    100
)
```

Simple but useful.

Tests I'd Remove

These aren't really box selector tests:

❌ Identical products all placed in same box

Already covered by:

`test_packing_engine.py`

That's packing logic.

❌ Newly added cheap box preferred

Covered by:

`returns cheapest fitting box`

Same scenario.

No need for a separate test.

Final Test List

If I were reviewing your assignment, I'd want exactly these:

```
test_returns_cheapest_fitting_box()
test_skips_invalid_cheapest_box()
test_returns_none_when_no_box_fits()
test_empty_order_returns_none()
test_invalid_order_id()
test_cheapest_among_multiple_valid_boxes()
test_response_contains_expected_fields()
test_volume_utilization_percentage_valid()
```

That's **8 high-value tests** that directly verify the responsibilities of `box_selector.py` without duplicating what you've already tested in `packing_engine.py`.

Then for `test_api.py`, keep it even smaller:

```
test_recommend_box_success()
test_order_not_found()
test_no_recommendation_available()
```

That gives you a very balanced testing strategy:

- **Packing Engine:** ~15 tests
- **Box Selector:** ~8 tests
- **API:** ~3 tests

Around **25-30 total tests**, which is excellent for this assignment.

This is better, but there are still **2 things I would change before submitting**.

---

1. Fix the 404 Test

Current:

```
with self.assertRaises(Exception):
    recommend_box(9999)
```

This is too broad.

If tomorrow your code accidentally raises:

```
TypeError
ValueError
AttributeError
```

the test still passes.

Use:

```
from django.http import Http404
def test_nonexistent_order_raises_404(self):
    with self.assertRaises(Http404):
        recommend_box(9999)
```

Much more precise.

---

2. Remove `test_expensive_box_not_chosen_when_cheaper_fits`

This test is essentially testing the same thing as:

```
test_prefers_cheaper_box_over_larger()
```

Both verify:

```
choose cheapest fitting box
```

You're not getting additional coverage.

I'd replace it with something more valuable.

Better Test: Quantity Affects Selection

This validates that your selector correctly uses:

```
item.quantity
```

for weight and volume calculations.

Example:

```
def test_quantity_affects_box_selection(self):
    order = self._create_order([
```

```
        (self.product, 5)
    ])
    result = recommend_box(order.id)
    self.assertIsNotNone(result)
    self.assertNotEqual(
        result["box_id"],
        self.small_box.id
    )
```

Even better if you calculate exactly which box should be selected.

This test verifies:

```
total_volume
total_weight
```

are multiplied by quantity.

That's much more valuable than re-testing cheapest-box behavior.

Optional: Weight Filter Test

This is another good selector-specific test.

```
def test_skips_box_with_insufficient_weight_capacity(self):
```

Create:

```
heavy = Product.objects.create(
    name="Heavy",
    length=5,
    width=5,
    height=5,
    weight=1500,
)
```

Now:

```
Small Box max_weight = 1000
Medium Box max_weight = 3000
```

Expected:

```
result["box_id"] == self.medium_box.id
```

This directly validates:

```
max_weight__gte=total_weight
```

inside `recommend_box()`.

Final Version I'd Submit

Keep

✅ `test_returns_none_for_impossible_product`

✅ `test_returns_none_for_empty_order`

✅ `test_prefers_cheaper_box_over_larger`

✅ `test_selects_larger_box_when_smaller_too_small`

✅ `test_volume_utilization_is_reasonable`

✅ `test_returns_expected_response_structure`

✅ `test_nonexistent_order_raises_404`

Add

✅ `test_quantity_affects_box_selection`

✅ `test_skips_box_with_insufficient_weight_capacity`

Remove

❌ `test_expensive_box_not_chosen_when_cheaper_fits`

After those changes, your `test_box_selector.py` would be around **9.5/10** and focused entirely on the responsibilities of the selector layer rather than duplicating packing-engine tests.

This is exactly the kind of output you want to see before submitting:

```
Found 64 test(s).
Ran 64 tests in 0.230s
OK
```

What This Tells a Reviewer

✅ You didn't just write code; you verified it.

✅ Models, services, and APIs are being tested.

✅ No failing edge cases in your current test suite.

✅ The project is stable enough to be run on another machine.

For the Assignment README

I'd add a section like:

Testing

The project includes comprehensive automated tests covering:

- Packing engine logic
    - Product orientation generation
    - Void splitting
    - Physical fit checks
    - Weight constraints
    - Multi-product and multi-quantity packing
- Box selection service
    - Cheapest valid box selection
    - Weight and dimension filtering
    - Empty orders
    - Invalid order handling
    - Response structure validation
- API endpoints
    - Successful recommendations
    - Error handling
    - Response validation

Running Tests

```
python manage.py test
```

Current test results:

```
Found 64 test(s).
Ran 64 tests in 0.230s

OK
```

What I'd Do Before Submission

Run:

```
git status
```

Verify:

- No `.env`
- No `db.sqlite3`
- No `__pycache__/`
- No `venv/`

README Should Contain

1. Project Overview
2. Architecture Diagram

3. Design Decisions
4. Packing Algorithm Explanation
5. API Endpoints
6. Test Instructions
7. Assumptions & Limitations

One More Thing

Since the assignment explicitly says:

> "You may use AI tools, but your submission must clearly show your own thinking."

Add a **Design Decisions** section explaining:

- Why you separated `packing_engine.py` and `box_selector.py`
- Why you use a greedy 3D packing heuristic
- Why boxes are sorted by cost
- Why weight and volume pre-checks are performed before packing

That section often matters more than another 20 lines of code because it demonstrates engineering judgment.

Current State Assessment

Area	Rating
Models	9/10
Service Layer	9.5/10
Packing Logic	9/10
API Design	9/10
Tests	10/10
Architecture	10/10

For a Django hiring assignment, this is a strong submission. At this point, I'd focus on polishing the README and ensuring the repository is clean rather than adding more features.

Link : https://chatgpt.com/share/6a3c3007-0cf8-83e8-ba27-52ba10e865ef

---